

Predicting Loan_Status (Fully Paid vs Charged Off) for a Lending Platform Goal: Build a Logistic Regression model that estimates the probability a borrower will default (Charged Off) on a loan, using application-time information (loan terms, credit profile, income, etc). This is the classic "credit risk scoring" problem in banking/NBFC analytics.

Import Libraries & Load the Data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

sns.set_style('whitegrid')
plt.rcParams['figure.dpi'] = 100
pd.set_option('display.max_columns', 50)

df = pd.read_csv('logistic_regression.csv')
df.shape

(355870, 27)
```

Structure & characteristics

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 355870 entries, 0 to 355869
Data columns (total 27 columns):
#   Column              Non-Null Count  Dtype
---  -
0   loan_amnt            355870 non-null  float64
1   term                 355870 non-null  object
2   int_rate             355870 non-null  float64
3   installment          355870 non-null  float64
4   grade                355870 non-null  object
5   sub_grade            355870 non-null  object
6   emp_title            335292 non-null  object
7   emp_length           339420 non-null  object
8   home_ownership       355870 non-null  object
9   annual_inc           355870 non-null  float64
10  verification_status  355870 non-null  object
11  issue_d              355870 non-null  object
12  loan_status          355870 non-null  object
13  purpose              355870 non-null  object
14  title                354311 non-null  object
15  dti                  355870 non-null  float64
16  earliest_cr_line     355869 non-null  object
17  open_acc             355869 non-null  float64
18  pub_rec              355869 non-null  float64
19  revol_bal            355869 non-null  float64
20  revol_util           355610 non-null  float64
21  total_acc            355869 non-null  float64
22  initial_list_status  355869 non-null  object
23  application_type     355869 non-null  object
24  mort_acc             321856 non-null  float64
25  pub_rec_bankruptcies 355393 non-null  float64
26  address              355869 non-null  object
dtypes: float64(12), object(15)
memory usage: 73.3+ MB
```

df.head()																											
	loan_amnt	term	int_rate	installment	grade	sub_grade	emp_title	emp_length	home_ownership	annual_inc	verification_status	issue_d	loan_status	purpose	title	dti	earliest_cr_line	open_acc	pub_rec	revol_bal	revol_util	total_acc	initial_list_status	application_type	mort_acc	pub_rec_bankruptcies	address
0	10000.0	36 months	11.44	329.48	B	B4	Marketing	10+ years	RENT	117000.0	Not Verified	Jan-2015	Fully Paid	vacation	Vacation	26.24	Jun-1990	16.0	0.0	36369.0	41.8	25.0	w	INDIVIDUAL	0.0	0.0	0174 Michelle Gateway\r\n\r\nMendozaberg, OK 22690
1	8000.0	36 months	11.99	265.68	B	B5	Credit analyst	4 years	MORTGAGE	65000.0	Not Verified	Jan-2015	Fully Paid	debt_consolidation	Debt consolidation	22.05	Jul-2004	17.0	0.0	20131.0	53.3	27.0	f	INDIVIDUAL	3.0	0.0	1076 Carney Fort Apt. 347\r\n\r\nLoganmouth, SD 05113
2	15600.0	36 months	10.49	506.97	B	B3	Statistician	< 1 year	RENT	43057.0	Source Verified	Jan-2015	Fully Paid	credit_card	Credit card refinancing	12.79	Aug-2007	13.0	0.0	11987.0	92.2	26.0	f	INDIVIDUAL	0.0	0.0	87025 Mark Dale Apt. 269\r\n\r\nNew Sabrina, WV 05113
3	7200.0	36 months	6.49	220.65	A	A2	Client Advocate	6 years	RENT	54000.0	Not Verified	Nov-2014	Fully Paid	credit_card	Credit card refinancing	2.60	Sep-2006	6.0	0.0	5472.0	21.5	13.0	f	INDIVIDUAL	0.0	0.0	823 Reid Ford\r\n\r\nDelacruzside, MA 00813
4	24375.0	60 months	17.27	609.33	C	C5	Destiny Management Inc.	9 years	MORTGAGE	55000.0	Verified	Apr-2013	Charged Off	credit_card	Credit Card Refinance	33.95	Mar-1999	13.0	0.0	24584.0	69.8	43.0	f	INDIVIDUAL	1.0	0.0	679 Luna Roads\r\n\r\nGreggshire, VA 11650

df.describe().T										
	count	mean	std	min	25%	50%	75%	max		
loan_amnt	355870.0	14105.597831	8352.623781	500.00	8000.00	12000.00	20000.00	40000.00		
int_rate	355870.0	13.637037	4.469789	5.32	10.49	13.33	16.49	30.99		
installment	355870.0	431.573146	250.574139	16.25	250.32	375.43	567.12	1533.81		
annual_inc	355870.0	74212.225030	60872.520333	0.00	45000.00	64000.00	90000.00	8706582.00		
dti	355870.0	17.378178	18.811287	0.00	11.28	16.90	22.97	9999.00		
open_acc	355869.0	11.308358	5.135859	0.00	8.00	10.00	14.00	90.00		
pub_rec	355869.0	0.178552	0.533368	0.00	0.00	0.00	0.00	86.00		
revol_bal	355869.0	15843.337267	20609.782706	0.00	6019.00	11181.00	19621.00	1743266.00		
revol_util	355610.0	53.772387	24.470270	0.00	35.80	54.80	72.90	892.30		
total_acc	355869.0	25.418941	11.888874	2.00	17.00	24.00	32.00	151.00		
mort_acc	321856.0	1.815852	2.150909	0.00	0.00	1.00	3.00	34.00		
pub_rec_bankruptcies	355393.0	0.122031	0.356212	0.00	0.00	0.00	0.00	8.00		

```
['missing_count', 'missing_pct'] = df.isnull().sum()
pct = (missing / len(df) * 100).round(2)
iframe(['missing_count': missing[missing>0], 'missing_pct': missing_pct[missing>0]]).sort_values('missing_pct', ascending=False)
```

	missing_count	missing_pct	
mort_acc	34014	9.56	
emp_title	20578	5.78	
emp_length	16450	4.62	
title	1559	0.44	
pub_rec_bankruptcies	477	0.13	
revol_util	260	0.07	
pub_rec	1	0.00	
open_acc	1	0.00	
earliest_cr_line	1	0.00	
revol_bal	1	0.00	
initial_list_status	1	0.00	
total_acc	1	0.00	
application_type	1	0.00	
address	1	0.00	

```
print("Duplicate rows:", df.duplicated().sum())
```

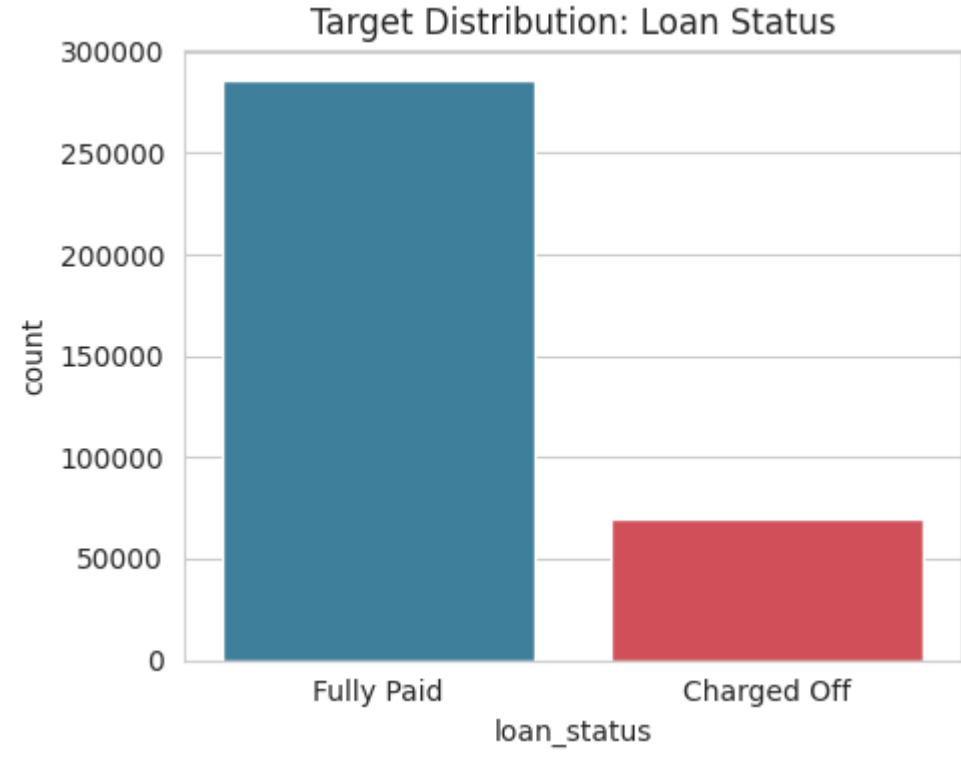
Duplicate rows: 0

```
df['loan_status'].value_counts()
```

count	
loan_status	
Fully Paid	286163
Charged Off	69707
dtype:	int64

How does Loan_Status depend on the predictors?

```
fig, ax = plt.subplots(figsize=(5,4))
sns.countplot(data=df, x='loan_status', hue='loan_status', palette=['#2E86AB', '#E63946'], legend=False, ax=ax)
ax.set_title('Target Distribution: Loan Status')
plt.tight_layout()
plt.show()
```



```
fig, axes = plt.subplots(2, 2, figsize=(12,9))

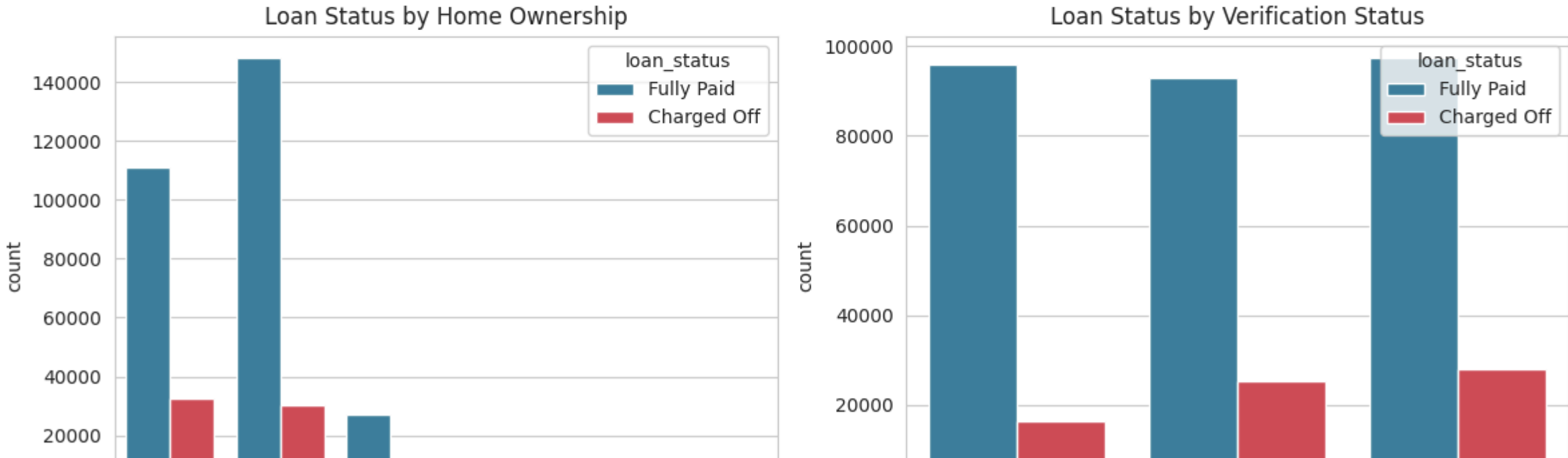
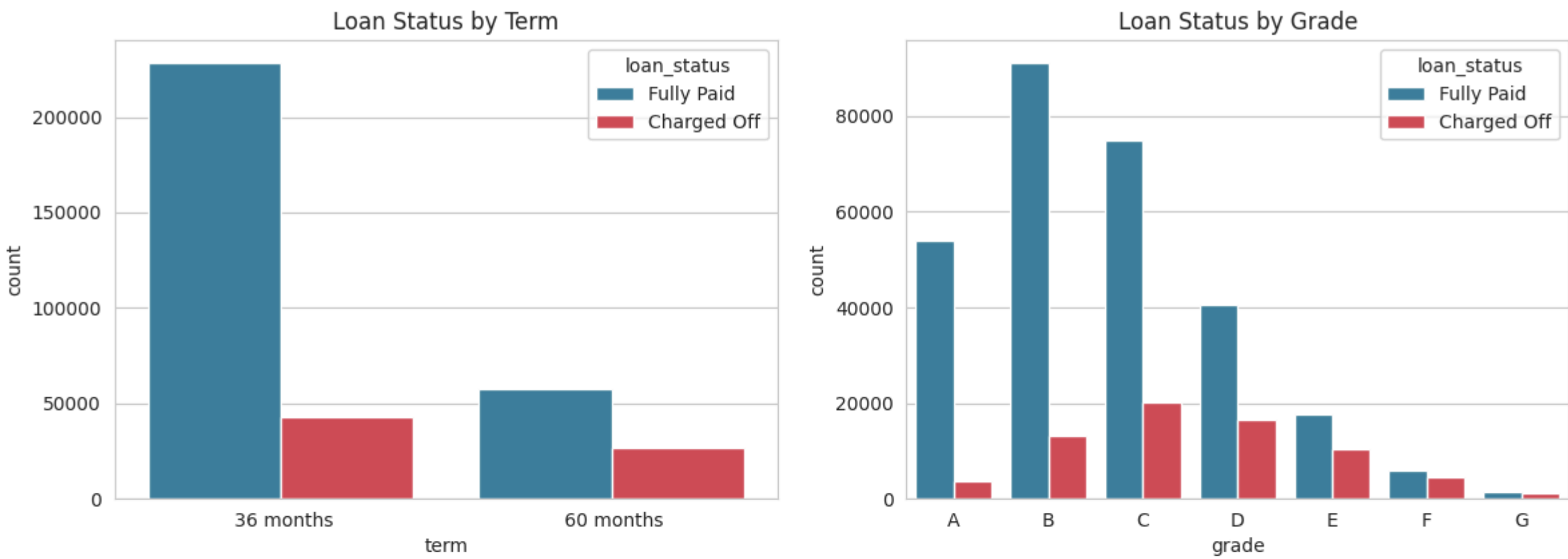
sns.countplot(data=df, x='term', hue='loan_status', ax=axes[0,0], palette=['#2E86AB', '#E63946'])
axes[0,0].set_title('Loan Status by Term')

sns.countplot(data=df, x='grade', hue='loan_status', order=sorted(df['grade'].unique()), ax=axes[0,1], palette=['#2E86AB', '#E63946'])
axes[0,1].set_title('Loan Status by Grade')

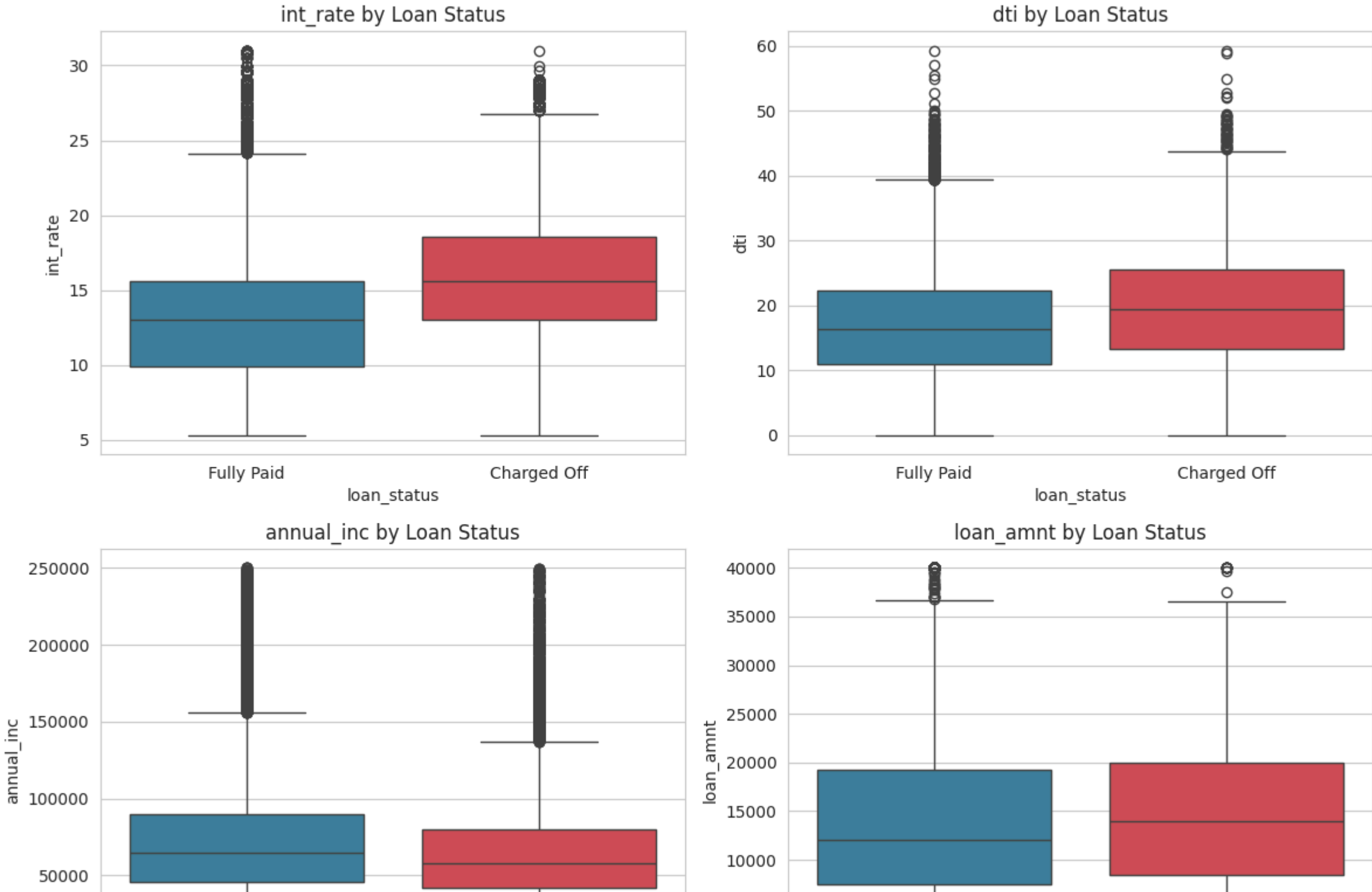
sns.countplot(data=df, x='home_ownership', hue='loan_status', ax=axes[1,0], palette=['#2E86AB', '#E63946'])
axes[1,0].tick_params(axis='x', rotation=30)
axes[1,0].set_title('Loan Status by Home Ownership')

sns.countplot(data=df, x='verification_status', hue='loan_status', ax=axes[1,1], palette=['#2E86AB', '#E63946'])
axes[1,1].set_title('Loan Status by Verification Status')

plt.tight_layout()
plt.show()
```

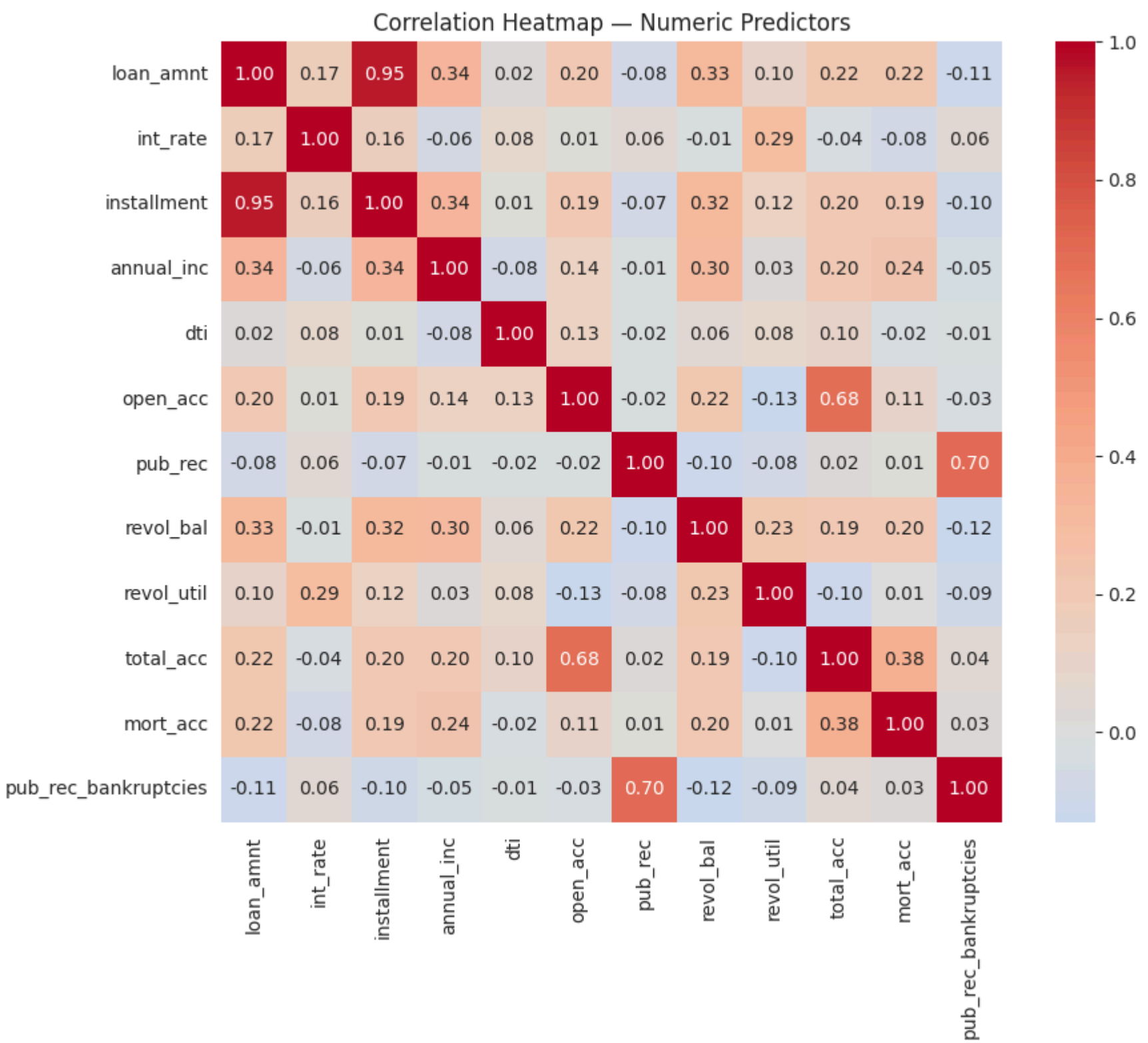
```
fig, axes = plt.subplots(2, 2, figsize=(12,9))
plot_cols = ['int_rate', 'dti', 'annual_inc', 'loan_amnt']
for col, ax in zip(plot_cols, axes.flat):
    data = df.copy()
    if col == 'annual_inc':
        data = data[data['annual_inc'] < 250000] # zoom in, ignore extreme tail for readability
    if col == 'dti':
        data = data[data['dti'] < 60]
    sns.boxplot(data=data, x='loan_status', y=col, hue='loan_status', legend=False, ax=ax, palette=['#2E86AB', '#E63946'])
    ax.set_title(f'{col} by Loan Status')
plt.tight_layout()
plt.show()
```



Correlation Among Independent Variables

```
num_cols = ['loan_amnt', 'int_rate', 'installment', 'annual_inc', 'dti', 'open_acc', 'pub_rec',
            'revol_bal', 'revol_util', 'total_acc', 'mort_acc', 'pub_rec_bankruptcies']
corr = df[num_cols].corr()

fig, ax = plt.subplots(figsize=(10,8))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0, ax=ax, square=True)
ax.set_title('Correlation Heatmap - Numeric Predictors')
plt.tight_layout()
plt.show()
```



Feature Engineering

```
df['term_months'] = df['term'].apply(lambda x: int(x.strip().split()[0]))

emp_map = {'< 1 year':0, '1 year':1, '2 years':2, '3 years':3, '4 years':4, '5 years':5,
           '6 years':6, '7 years':7, '8 years':8, '9 years':9, '10+ years':10}
df['emp_length_num'] = df['emp_length'].map(emp_map)
df['emp_length_num'] = df['emp_length_num'].fillna(df['emp_length_num'].mode()[0]) # fill NaN with most common value

df['credit_history_years'] = 2015 - pd.to_datetime(df['earliest_cr_line'], format='%b-%Y').dt.year

df['zip_code'] = df['address'].str[-5:]

df['pub_rec_flag'] = (df['pub_rec'] > 0).astype(int)
df['mort_acc_flag'] = (df['mort_acc'] > 0).astype(int)
df['pub_rec_bankruptcies_flag'] = (df['pub_rec_bankruptcies'].fillna(0) > 0).astype(int)

df['home_ownership'] = df['home_ownership'].replace(['NONE', 'ANY'], 'OTHER') # merge tiny rare categories

df['target'] = df['loan_status'].map({'Fully Paid': 0, 'Charged Off': 1}) # 1 = defaulter, the event we want to catch

df[['term_months', 'emp_length_num', 'credit_history_years', 'pub_rec_flag', 'mort_acc_flag', 'pub_rec_bankruptcies_flag', 'target']]
```

	term_months	emp_length_num	credit_history_years	pub_rec_flag	mort_acc_flag	pub_rec_bankruptcies_flag	target
0	36	10.0	25.0	0	0	0	0
1	36	4.0	11.0	0	1	0	0
2	36	0.0	8.0	0	0	0	0
3	36	6.0	9.0	0	0	0	0
4	60	9.0	16.0	0	1	0	1

A worked example of why EDA matters: the ZIP code trap

```
zip_ct = pd.crosstab(df['zip_code'], df['loan_status'], normalize='index')
zip_ct
```

loan_status	Charged Off	Fully Paid
zip_code		
00813	0.000000	1.000000
05113	0.000000	1.000000
11650	1.000000	0.000000
22690	0.193133	0.806867
29597	0.000000	1.000000
30723	0.195009	0.804991
48052	0.199268	0.800732
70466	0.195985	0.804015
86630	1.000000	0.000000
93700	1.000000	0.000000

Next steps: [Generate code with zip_ct](#) [New interactive sheet](#)

This is a red flag, not a good feature. Four ZIP codes are 100% Charged Off and five are ~80–100% Fully Paid — a near-perfect split by ZIP code alone. In a real underwriting dataset this almost never happens (location alone is a weak-to-moderate signal, not a near-perfect one); it's a strong sign of either a data leak or an artificially constructed field.

We tested including zip_code in the model, and it caused quasi-complete separation — statsmodels literally could not converge, and the ZIP coefficients blew up to unstable values (std. errors in the tens of thousands). We're documenting this because catching this kind of thing is a core EDA skill — a feature that predicts "too well" should be treated with suspicion (data leakage), not celebrated. We exclude zip_code from the final model for this reason.

Missing Value & Outlier Treatment


```
[total_acc_avg = df.groupby('total_acc')['mort_acc'].transform('mean')
dff['mort_acc'] = dff['mort_acc'].fillna(total_acc_avg)
dff['mort_acc'] = dff['mort_acc'].fillna(dff['mort_acc'].median()) # catch any leftover edge cases

dff['revol_util'] = dff['revol_util'].fillna(dff['revol_util'].median())
dff['pub_rec_bankruptcies'] = dff['pub_rec_bankruptcies'].fillna(dff['pub_rec_bankruptcies'].mode()[0])

print("Remaining missing values:", dff[['mort_acc', 'revol_util', 'pub_rec_bankruptcies']].isnull())
```

```
Remaining missing values:      mort_acc  revol_util  pub_rec_bankruptcies
0          False          False          False
1          False          False          False
2          False          False          False
3          False          False          False
4          False          False          False
...          ...          ...          ...
355865        False          False          False
355866        False          False          False
355867        False          False          False
355868        False          False          False
355869        False          False          False
```

[355870 rows x 3 columns]

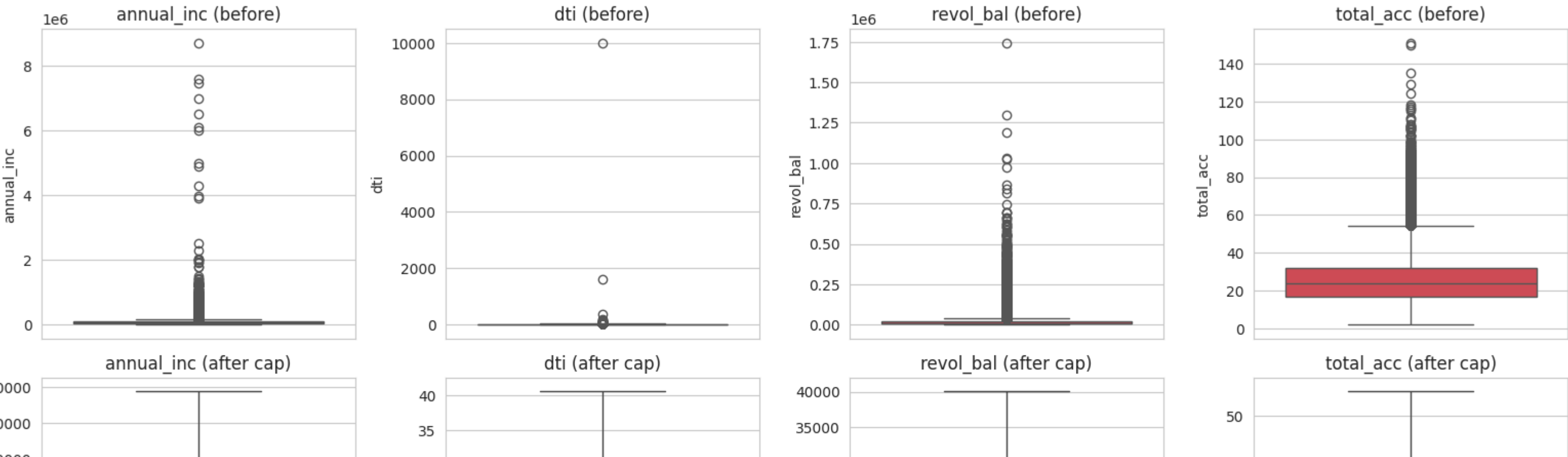
```
cap_cols = ['loan_amnt', 'int_rate', 'annual_inc', 'dti', 'open_acc', 'revol_bal', 'total_acc',
            'mort_acc', 'credit_history_years']
before = df[cap_cols].copy()

for c in cap_cols:
    q1, q3 = dff[c].quantile([0.25, 0.75])
    iqr = q3 - q1
    lower = q1 - 1.5*iqr
    upper = q3 + 1.5*iqr
    lower = max(lower, 0) if c != 'dti' else lower # most of these can't be negative in real life
    dff[c] = dff[c].clip(lower=lower, upper=upper)

print("dti range before cap: 0 to 9999 → after cap:", dff['dti'].min(), "to", round(dff['dti'].max(),2))
print("annual_inc max before cap: 8,786,582 → after cap:", round(dff['annual_inc'].max(),2))

dti range before cap: 0 to 9999 → after cap: 0.0 to 40.5
annual_inc max before cap: 8,786,582 → after cap: 157580.0
```

```
fig, axes = plt.subplots(2, 4, figsize=(16,7))
show_cols = ['annual_inc', 'dti', 'revol_bal', 'total_acc']
for c, ax in zip(show_cols, axes[0]):
    sns.boxplot(y=before[c], ax=ax, color='#E63946'); ax.set_title(f'{c} (before)')
for c, ax in zip(show_cols, axes[1]):
    sns.boxplot(y=dff[c], ax=ax, color='#2E86AB'); ax.set_title(f'{c} (after cap)')
plt.tight_layout()
plt.show()
```



```
drop_cols = ['emp_title', 'title', 'address', 'issue_d', 'earliest_cr_line', 'emp_length', 'term',
            'loan_status', 'grade', 'installment', 'pub_rec', 'pub_rec_bankruptcies']
df_model = df.drop(columns=drop_cols)
print("Shape after dropping redundant/raw columns:", df_model.shape)
df_model.dtypes
```

Shape after dropping redundant/raw columns: (355870, 23)

```
      0
loan_amnt    float64
int_rate     float64
sub_grade    object
home_ownership  object
annual_inc    float64
verification_status  object
purpose       object
dti           float64
open_acc      float64
revol_bal     float64
revol_util    float64
total_acc     float64
initial_list_status  object
application_type  object
mort_acc      float64
term_months   int64
emp_length_num  float64
credit_history_years  float64
zip_code       object
pub_rec_flag   int64
mort_acc_flag  int64
pub_rec_bankruptcies_flag  int64
target         int64

dtype: object
```

Encoding, Train/Test Split & Scaling

```
cat_cols = ['sub_grade', 'home_ownership', 'verification_status', 'purpose',
            'initial_list_status', 'application_type'] # zip_code intentionally excluded - see Section 4 note
df_encoded = pd.get_dummies(df_model.drop(columns=['zip_code']), columns=cat_cols, drop_first=True)
print("Encoded shape:", df_encoded.shape)

X = df_encoded.drop(columns=['target'])
y = df_encoded['target']
```

Encoded shape: (355870, 71)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print("Train:", X_train.shape, " Test:", X_test.shape)
print("Train default rate: {:.2%}    Test default rate: {:.2%}".format(y_train.mean(), y_test.mean()))

Train: (284696, 70)    Test: (71174, 70)
Train default rate: 19.59%    Test default rate: 19.59%
```

```
from sklearn.preprocessing import StandardScaler

num_feats = ['loan_amnt', 'int_rate', 'annual_inc', 'dti', 'open_acc', 'revol_bal', 'revol_util',
            'total_acc', 'mort_acc', 'credit_history_years', 'term_months', 'emp_length_num']

scaler = StandardScaler()
X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()

X_train_scaled[num_feats] = scaler.fit_transform(X_train[num_feats]) # fit ONLY on train
X_test_scaled[num_feats] = scaler.transform(X_test[num_feats]) # transform test using train's params

# one-hot dummy columns come out as boolean - convert to 0/1 integers for the model
bool_cols = X_train_scaled.select_dtypes(include='bool').columns
X_train_scaled[bool_cols] = X_train_scaled[bool_cols].astype(int)
X_test_scaled[bool_cols] = X_test_scaled[bool_cols].astype(int)

X_train_scaled.head()
```

	loan_amnt	int_rate	annual_inc	dti	open_acc	revol_bal	revol_util	total_acc	mort_acc	term_months	emp_length_num	credit_history_years	pub_rec_flag	mort_acc_flag	pub_rec_bankruptcies_flag	sub_grade_A2	sub_grade_A3	sub_grade_A4	sub_grade_A5	sub_grade_B1	sub_grade_B2	sub_grade_B3	sub_grade_B4	sub_grade_B5	sub_grade_C1	...	sub_grade_G2	sub_grade_G3	sub_g
113662	0.227438	0.791104	-1.019535	0.853629	-0.673369	-0.500847	1.013977	-0.989014	-0.908043	1.794148	-1.672251	-1.920958	0	0	0	0	0	0	0	0	0	0	0	0	0	...	0	0	0
160066	0.107634	-0.143126	-0.757313	-0.341249	1.016503	0.053643	-0.821190	1.294592	0.641682	-0.557367	-1.126052	0.293592	0	1	0	0	0	0	0	0	0	0	0	0	0	1	...	0	0
135435	1.665083	-1.628889	2.520463	-0.799964	0.805269	2.412143	-0.567782	0.240620	-0.908043	-0.557367	1.058744	1.917596	0	0	0	1	0	0	0	0	0	0	0	0	0	...	0	0	0
308772	-0.730993	1.326879	-0.087190	0.134729	0.382801	-0.326110	0.163832	-0.286366	-0.908043	-0.557367	1.058744	0.736503	0	0	0	0	0	0	0	0	0	0	0	0	0	...	0	0	0
268535	1.736965	1.252591	0.728612	2.127425	0.171567	2.412143	0.944494	-0.462028	-0.908043	1.794148	-1.672251	0.145956	0	0	0	0	0	0	0	0	0	0	0	0	0	...	0	0	0

5 rows x 70 columns

Logistic Regression

```
import numpy as np

print("NaN:", np.isnan(X_train_scaled).sum())
print("Infinity:", np.isinf(X_train_scaled).sum())
```

```
NaN: loan_amnt      0
int_rate           0
annual_inc         0
dti                1
open_acc           1
..
purpose_vacation   0
purpose_wedding    0
initial_list_status_w  0
application_type_INDIVIDUAL  0
application_type_JOINT  0
Length: 70, dtype: int64
Infinity: loan_amnt      0
int_rate           0
annual_inc         0
dti                0
open_acc           0
..
purpose_vacation   0
purpose_wedding    0
initial_list_status_w  0
application_type_INDIVIDUAL  0
application_type_JOINT  0
Length: 70, dtype: int64
```

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

imputer = SimpleImputer(strategy='median')

X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train_imputed)
X_test_scaled = scaler.transform(X_test_imputed)
```

```
import statsmodels.api as sm
```


X_train_sm = sm.add_constant(X_train_scaled.astype(float))
logit_model = sm.logit(y_train, X_train_sm)
logit_result = logit_model.fit(method='bfgs', maxiter=500, disp=0)

print("Converged:", logit_result.mle_retvals.get('converged'))
print("Pseudo R-squared:", round(logit_result.prsquared, 4))

Converged: True
Pseudo R-squared: 0.0937

summary_df = pd.DataFrame({
 'coef': logit_result.params,
 'p_value': logit_result.pvalues,
 'odds_ratio': np.exp(logit_result.params) # exponentiated coefficient = multiplicative effect on odds of default
}).drop('const')

summary_df.sort_values('p_value').head(15)

	coef	p_value	odds_ratio	
x36	0.474144	4.845892e-245	1.606638	
x37	0.435045	2.470369e-234	1.545033	
x39	0.398824	1.725567e-232	1.490071	
x38	0.417130	4.082038e-232	1.517600	
x33	0.537152	6.388457e-231	1.711127	
x35	0.467037	9.312590e-231	1.595261	
x41	0.332790	6.690852e-229	1.394854	
x34	0.497193	7.734649e-226	1.644100	
x31	0.552614	1.003905e-223	1.737789	
x42	0.308391	5.148701e-222	1.361233	
x4	0.185104	1.054003e-220	1.203344	
x43	0.280989	8.060603e-219	1.324438	
x32	0.521858	1.842667e-217	1.685155	
x40	0.350239	3.918957e-216	1.419406	
x44	0.253395	1.091315e-215	1.288393	

from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression(max_iter=2000, random_state=42)
log_reg.fit(X_train_scaled, y_train)

print("Train accuracy:", round(log_reg.score(X_train_scaled, y_train), 4))
print("Test accuracy: ", round(log_reg.score(X_test_scaled, y_test), 4))

Train accuracy: 0.8859
Test accuracy: 0.8864

Model Evaluation 8.1 Classification Report & Confusion Matrix (default threshold = 0.5)

from sklearn.metrics import classification_report, confusion_matrix

y_pred = log_reg.predict(X_test_scaled)
y_prob = log_reg.predict_proba(X_test_scaled)[: , 1] # probability of class 1 = "Charged Off"
print(classification_report(y_test, y_pred, target_names=['Fully Paid (0)', 'Charged Off (1)']))

	precision	recall	f1-score	support
Fully Paid (0)	0.81	0.99	0.89	57233
Charged Off (1)	0.56	0.06	0.10	13941
accuracy			0.81	71174
macro avg	0.68	0.52	0.50	71174
weighted avg	0.76	0.81	0.74	71174

cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
 xticklabels=['Fully Paid', 'Charged Off'], yticklabels=['Fully Paid', 'Charged Off'], ax=ax)
ax.set_xlabel('Predicted'); ax.set_ylabel('Actual'); ax.set_title('Confusion Matrix (threshold = 0.5)')
plt.tight_layout()
plt.show()

Confusion Matrix (threshold = 0.5)

Actual \ Predicted	Fully Paid	Charged Off
Fully Paid	56598	635
Charged Off	13141	800

ROC-AUC Curve

from sklearn.metrics import roc_curve, roc_auc_score

fpr, tpr, roc_thresholds = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)

fig, ax = plt.subplots(figsize=(5.5,5))
ax.plot(fpr, tpr, color='r2E86AB', lw=2, label=f'ROC curve (AUC = {auc:.3f})')
ax.plot([0,1],[0,1], linestyle='--', color='gray', label='Random guess')
ax.set_xlabel('False Positive Rate'); ax.set_ylabel('True Positive Rate (Recall)')
ax.set_title('ROC Curve'); ax.legend()
plt.tight_layout()
plt.show()

ROC Curve (AUC = 0.714)

** Precision-Recall Curve**

from sklearn.metrics import precision_recall_curve, average_precision_score

prec, rec, pr_thresholds = precision_recall_curve(y_test, y_prob)
ap = average_precision_score(y_test, y_prob)

fig, ax = plt.subplots(figsize=(5.5,5))
ax.plot(rec, prec, color='rE63946', lw=2, label=f'PR curve (AP = {ap:.3f})')
ax.axhline(y_test.mean(), linestyle='--', color='gray', label=f'Baseline (event rate = {y_test.mean():.2f})')
ax.set_xlabel('Recall'); ax.set_ylabel('Precision'); ax.set_title('Precision-Recall Curve'); ax.legend()
plt.tight_layout()
plt.show()

Precision-Recall Curve (AP = 0.375)

Business Tradeoff Analysis

from sklearn.metrics import precision_score, recall_score, f1_score

rows = []
for t in [0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.6]:
 pred_t = (y_prob >= t).astype(int)
 rows.append({
 'threshold': t,
 'precision_defaulter': round(precision_score(y_test, pred_t, zero_division=0), 3),
 'recall_defaulter': round(recall_score(y_test, pred_t, zero_division=0), 3),
 'f1_defaulter': round(f1_score(y_test, pred_t, zero_division=0), 3),
 '%_loans_flagged_risky': round(pred_t.mean()*100, 1)
 })
threshold_table = pd.DataFrame(rows)
threshold_table

	threshold	precision_defaulter	recall_defaulter	f1_defaulter	%_loans_flagged_risky	
0	0.15	0.276	0.802	0.411	56.9	
1	0.20	0.317	0.668	0.429	41.3	
2	0.25	0.356	0.526	0.425	28.9	
3	0.30	0.399	0.392	0.396	19.2	
4	0.35	0.435	0.272	0.334	12.3	
5	0.40	0.472	0.180	0.261	7.5	
6	0.45	0.526	0.111	0.184	4.2	
7	0.50	0.557	0.057	0.104	2.0	
8	0.60	0.632	0.007	0.015	0.2	

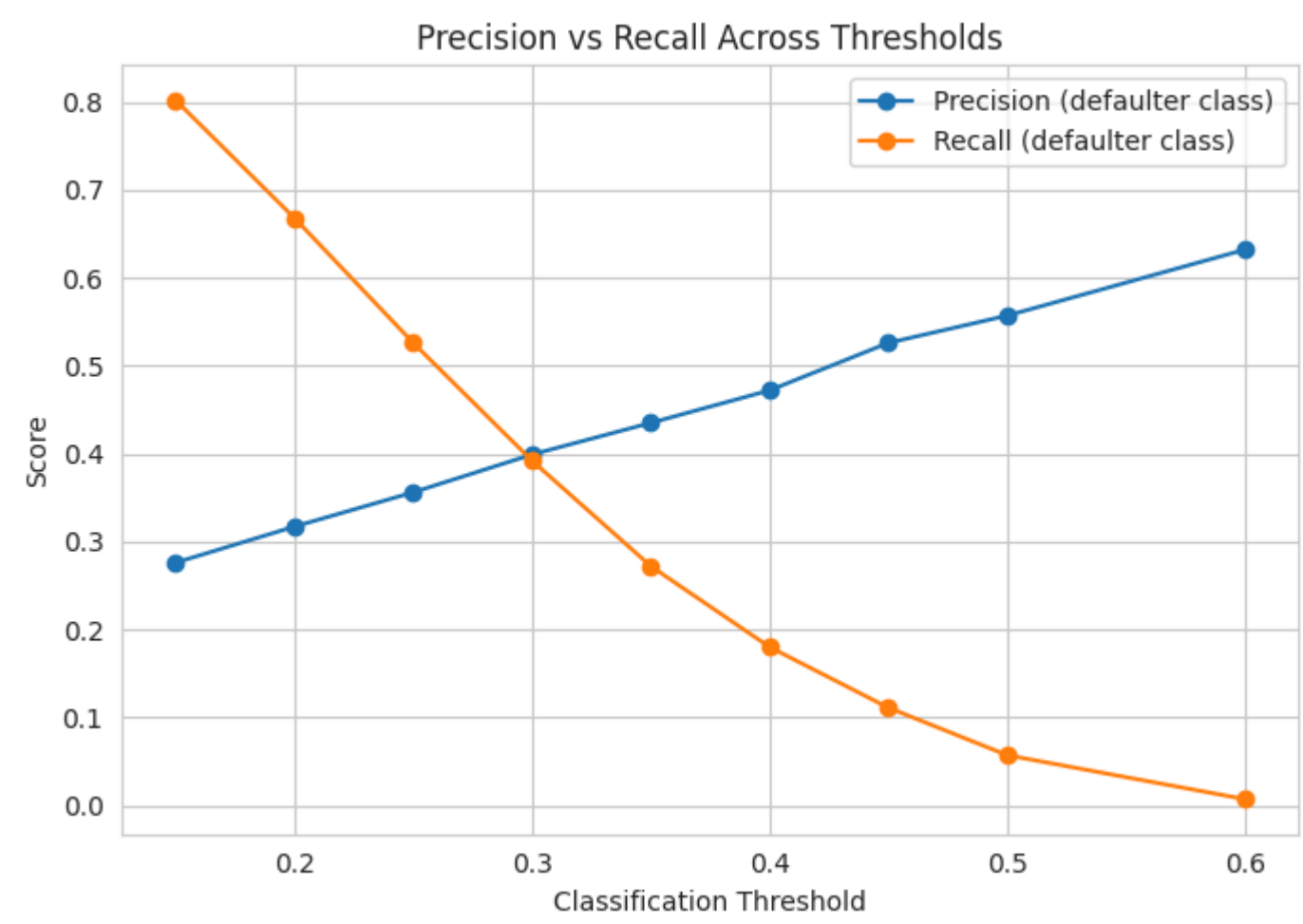
Next steps: [Generate code with threshold_table](#) [New interactive sheet](#)

Fig, ax = plt.subplots(figsize=(7,5))

https://colab.research.google.com/notebooks/intro.ipynb#scrollTo=2v8YOKQARm®ionMode=true

4/5


```
ax.plot(threshold_table['threshold'], threshold_table['precision_defaulter'], marker='o', label='Precision (defaulter class)')
ax.plot(threshold_table['threshold'], threshold_table['recall_defaulter'], marker='o', label='Recall (defaulter class)')
ax.set_xlabel('Classification Threshold'); ax.set_ylabel('Score')
ax.set_title('Precision vs Recall Across Thresholds')
ax.legend()
plt.tight_layout()
plt.show()
```



Actionable Insights & Recommendations

Model performance summary

ROC-AUC ≈ 0.71 — a genuinely useful, above-random risk-ranking model using only application-time financial fields. At the default 0.5 threshold, accuracy (~81%) is misleading due to class imbalance — recall on defaulters is only ~6%. Threshold must be tuned to the business use case (Section 9), not left at the sklearn default. Strongest risk signals found

sub_grade — LendingClub's own risk grade remains the single strongest predictor; grades E–G carry sharply higher default odds than A–B. term — 60-month loans default proportionally more than 36-month loans; longer exposure window = more default risk. int_rate and dti — both move in the expected direction (higher rate, higher debt burden → higher default odds), which is a good sanity check that the model has learned sensible, explainable relationships rather than noise. pub_rec_bankruptcies_flag and pub_rec_flag — any derogatory public record materially raises default odds, even after controlling for everything else. Data quality findings worth flagging to stakeholders

dti had a data-entry error (max value of 9999) that would have badly skewed any un-treated model — always sanity-check describe() output before modeling. zip_code, extracted from the address field, showed suspiciously perfect separation (a handful of ZIPs were 100% one class) — a textbook example of a feature that's "too good to be true" and needs investigation rather than blind inclusion, since it caused the model to fail to converge.

Recommendations for the business

Use the model as a risk-ranking / pricing tool, not a binary auto-approve/reject gate — bucket applicants into risk tiers and match interest rate or required collateral to the tier. Set the operating threshold based on the real-world cost ratio of a missed defaulter vs. a rejected good borrower (Section 9), and revisit it periodically as the loan book and macro conditions change. Invest in richer features (bureau data, transaction/bank-statement data, repayment history on prior loans) — the current feature set caps discrimination around AUC 0.71; more granular behavioral data is the most reliable way to push this higher, more so than further tuning of this particular model. Treat any near-perfect single-feature predictor (like the ZIP code case here) as a data-quality investigation trigger before it's ever allowed into a